claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_498b24af-d16\agent-a6c6cd802c953a48c.jsonl`
- SHA-256: `c3ac64af3d129f80a35cda8f3ce2a86e25021a2b1a7c2a35830bad9324873aaa`
- Source modified: `2026-06-21T11:12:49+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_498b24af-d16`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`

Transcript

1. user (2026-06-21T11:10:34.619Z)

Review the COMPUTE_DECOUPLED_SERVING M8 change just committed on branch feat/serving-m8-cost-ledger (HEAD). Inspect with: git diff origin/master...HEAD and read: backend/billing.py (pure cost math compute_cost_usd/apply_margin/to_inr); backend/infrastructure/database.py (_migrate_to_v6: cost columns on jobs + pricebook table seeded placeholder-v0 at 0.0; get_pricebook_rates, record_job_cost, get_job_cost, get_video_cost); backend/config/models.py (BillingConfig, default-OFF); backend/main.py (GET /api/jobs/{id}/cost + /api/videos/{id}/cost); backend/api/job_worker.py (_maybe_record_job_cost, gated); tests/test_billing.py + the cost tests in tests/test_api_endpoints.py. CONTRACT: DEFAULT-OFF ? billing.enabled=False records nothing (cost columns stay NULL = today); the seeded placeholder-v0 pricebook makes every cost 0; the v6 migration is additive-NULL + idempotent. USD is the source of truth, INR a display conversion. Report ONLY real, grounded issues with file:line. Empty findings if a lens finds nothing real.

LENS = COST CORRECTNESS / FINANCIAL MATH. (1) compute_cost_usd: is gemini_usd correctly a passthrough (added, NOT rate-multiplied)? Are missing amounts/rates safe (to 0)? Float rounding ? could a real bill drift (round at 6dp)? (2) get_pricebook_rates: does it correctly pick the gpu_type-specific gpu_seconds rate AND the gpu_type-empty wall/egress rates? What if a version has a gpu_seconds row but NOT for the requested gpu_type (no gpu cost, silently) ? right behaviour or a silent under-bill? (3) record_job_cost: bytes_out = int(egress_gb times 1e9) ? is the GB-to-bytes round-trip consistent with the egress_gb used for cost? (4) apply_margin clamps margin to >=0 ? correct (never bill below cost)? (5) to_inr at 2dp is display-only, USD source of truth? (6) get_video_cost: SUM cost_usd skipping NULLs, and is summing across re-ingest jobs the right semantic? Find real money-math errors or silent under/over-billing.

2. assistant (2026-06-21T11:10:36.925Z)

I'll review the M8 cost ledger change. Let me start by pulling the latest and inspecting the diff.

3. user (2026-06-21T11:10:40.373Z)

582522b feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF)
8cdf65c feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked) (#287)
b5af66e feat(serving): M5 ? truly-local profile per-profile startup checks + L4 profile-parity test (default-OFF) (#286)
---BRANCH---
feat/serving-m8-cost-ledger

4. user (2026-06-21T11:10:40.591Z)

```
 backend/api/job_worker.py          |  27 +++++++
 backend/billing.py                 |  60 ++++++++++++++
 backend/config/models.py           |  16 ++++
```

```
 backend/infrastructure/database.py | 132 +++++++++++++++++++++++++++++++-
 backend/main.py                    |  28 +++++++
 tests/test_api_endpoints.py        |  46 ++++++++++
 tests/test_async_jobs.py           |   8 +-
 tests/test_billing.py              | 157 ++++++++++++++++++++++++++++++++++++
 tests/test_database.py             |  28 ++++---
 tests/test_job_worker_loop.py      |   8 +-
 10 files changed, 490 insertions(+), 20 deletions(-)
```

5. assistant (2026-06-21T11:10:42.489Z)

Let me read the core files of the diff.

6. user (2026-06-21T11:10:43.597Z)

```
diff --git a/backend/billing.py b/backend/billing.py
new file mode 100644
index 0000000..c7f3445
--- /dev/null
+++ b/backend/billing.py
@@ -0,0 +1,60 @@
+"""Per-job cost ledger ? pure cost math (COMPUTE_DECOUPLED_SERVING M8, decision D8).
+
+``cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)``
+(the formula from the archived 05-COST-ATTRIBUTION design). Cost is computed + stored in
**USD**
+(matches GCP billing = one source of truth) and **displayed in INR** at a configurable FX rate
+(the Bangalore market). The pricebook is a versioned DB table
(``backend/infrastructure/database.py``);
+this module is the pure arithmetic over a resolved ``{resource: unit_price_usd}`` rates dict,
so it
+is unit-tested with zero IO and the DB layer owns the version/gpu_type lookup.
+
+**Default-OFF:** with the seeded ``placeholder-v0`` pricebook (every rate ``0.0``) every cost
is
+``0.0`` ? nothing user-facing changes until the owner inserts a real, calibrated pricebook
version.
+"""
+
+from __future__ import annotations
+
+from typing import Dict, Mapping, Tuple
+
+# Metered resource keys ? shared by the usage dict, the pricebook rows, and the breakdown.
+GPU_SECONDS = "gpu_seconds"     # GPU-active seconds (the big one) ? rate is per-second, per
gpu_type
+WALL_SECONDS = "wall_seconds"   # total wall time (separate from GPU so idle/sync isn't billed
as GPU)
+EGRESS_GB = "egress_gb"         # reel/byte egress in GB
+GEMINI_USD = "gemini_usd"       # provider-reported cost, ALREADY in USD (a passthrough, not
rate-×)
+
+# Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a USD
amount).
+_RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
+
+
+def compute_cost_usd(usage: Mapping[str, float],
+                     rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
+    """Return ``(cost_usd, breakdown)`` for one job.
+
+    ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps a
resource
+    key to its USD unit price (already resolved for this pricebook version + gpu_type by the DB
+    layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
```

```
multiplied).
+    Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total is
``0.0``.
+    """
+    breakdown: Dict[str, float] = {}
+    total = 0.0
+    for res in _RATE_MULTIPLIED:
+        amount = float(usage.get(res, 0.0) or 0.0)
+        rate = float(rates.get(res, 0.0) or 0.0)
+        cost = amount * rate
+        if amount or rate:                # record the line even at $0 so the breakdown is
transparent
+            breakdown[res] = round(cost, 6)
+        total += cost
+    gemini = float(usage.get(GEMINI_USD, 0.0) or 0.0)
+    if gemini:
+        breakdown[GEMINI_USD] = round(gemini, 6)
+    total += gemini
+    return round(total, 6), breakdown
+
+
+def apply_margin(cost_usd: float, margin: float) -> float:
+    """Resale price = ``cost_usd × (1 + margin)``. ``margin`` is clamped ? 0 (never below
cost)."""
+    return round(cost_usd * (1.0 + max(0.0, float(margin))), 6)
+
+
+def to_inr(cost_usd: float, fx_inr_per_usd: float) -> float:
+    """Display conversion: USD ? INR at the configured FX rate (2 dp, the display currency)."""
+    return round(float(cost_usd) * float(fx_inr_per_usd), 2)
```

7. user (2026-06-21T11:10:43.943Z)

```
diff --git a/backend/infrastructure/database.py b/backend/infrastructure/database.py
index ba7a59c..6ae4b87 100644
--- a/backend/infrastructure/database.py
+++ b/backend/infrastructure/database.py
@@ -4,6 +4,8 @@ import logging
 from contextlib import contextmanager
 from typing import List, Dict, Any, Optional, Tuple

+from backend import billing  # pure cost math (no backend imports ? no cycle)
+
 logger = logging.getLogger(__name__)

 class Database:
@@ -102,7 +104,10 @@ class Database:

             if version < 5:
                 self._migrate_to_v5(cursor)
-            # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA user_version =
6
+
+            if version < 6:
+                self._migrate_to_v6(cursor)
+            # future: if version < 7: cursor.execute("ALTER TABLE ..."); PRAGMA user_version =
7

             conn.commit()

@@ -284,6 +289,53 @@ class Database:
         """)
         cursor.execute("PRAGMA user_version = 5")

+    def _migrate_to_v6(self, cursor):
```

```
+        # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost columns on
+        # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records until
+        # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is stored
in
+        # USD (matches GCP billing = one source of truth); INR is a display conversion at a
+        # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD COLUMNs are
+        # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no served
change.
+        for col, typ in (
+            ("owner_id", "TEXT"),               # who pays (the non-owner user / tenant)
+            ("compute_backend", "TEXT"),        # local_gpu | cloud_run | ...
+            ("gpu_type", "TEXT"),               # L4 | T4 | ... (selects the per-second GPU
rate)
+            ("gpu_active_seconds", "REAL"),
+            ("wall_seconds", "REAL"),
+            ("bytes_out", "INTEGER"),           # egress
+            ("gemini_cost_usd", "REAL"),        # provider-reported, already USD
+            ("pricebook_version", "TEXT"),
+            ("cost_usd", "REAL"),               # computed total (USD = source of truth)
+            ("cost_breakdown_json", "TEXT"),    # {gpu_seconds: .., egress_gb: .., gemini_usd:
..}
+        ):
+            self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col} {typ}")
+        # Partial index keeps the NULL=no-cost fast path churn-free while making per-owner
billing cheap.
+        cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
+                       "WHERE owner_id IS NOT NULL")
+        # Versioned pricebook: one row per (version, resource, gpu_type). Rates change + differ
by
+        # gpu_type ? re-version (never mutate a shipped version, so a recorded cost stays
auditable).
+        cursor.execute("""
+            CREATE TABLE IF NOT EXISTS pricebook (
+                version TEXT NOT NULL,
+                resource TEXT NOT NULL,
+                gpu_type TEXT NOT NULL DEFAULT '',
+                unit_price_usd REAL NOT NULL DEFAULT 0.0,
+                currency TEXT NOT NULL DEFAULT 'USD',
+                effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
+            )
+        """)
+        cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
+                       "ON pricebook(version, resource, gpu_type)")
+        # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0 (nothing
+        # user-facing changes). The owner INSERTs a real, calibrated version (real GCP $/GPU-s,
+        # egress $/GB) and flips billing.pricebook_version to it ? never overwrite this seed.
+        for resource, gpu_type in (("gpu_seconds", "L4"), ("gpu_seconds", "T4"),
+                                   ("wall_seconds", ""), ("egress_gb", "")):
+            cursor.execute(
+                "INSERT OR IGNORE INTO pricebook (version, resource, gpu_type, unit_price_usd,
"
+                "currency) VALUES ('placeholder-v0', ?, ?, 0.0, 'USD')", (resource, gpu_type))
+        cursor.execute("PRAGMA user_version = 6")
+
     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
        """Register or update a video row WITHOUT touching its child segments.

@@ -547,6 +599,84 @@ class Database:
            d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
            return d

+    # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8) ----------------------------
+    def get_pricebook_rates(self, version: str, gpu_type: Optional[str] = None) -> Dict[str,
float]:
```

```
+        """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``. The per-
second
+        GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
``gpu_type=''``.
+        An unknown version ? ``{}`` (so every cost computes to 0.0)."""
+        gt = gpu_type or ""
+        rates: Dict[str, float] = {}
+        with self._connect() as conn:
+            rows = conn.cursor().execute(
+                "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE version = ?",
+                (version,)).fetchall()
+        for r in rows:
+            res, row_gt, price = r["resource"], (r["gpu_type"] or ""),
float(r["unit_price_usd"])
+            if res == billing.GPU_SECONDS:
+                if row_gt == gt:        # the GPU rate for THIS gpu_type
+                    rates[res] = price
+            else:
+                rates[res] = price      # wall/egress: gpu_type-independent
+        return rates
+
+    def record_job_cost(self, job_id: str, *, usage: Dict[str, float], pricebook_version: str,
+                        gpu_type: Optional[str] = None, compute_backend: Optional[str] = None,
+                        owner_id: Optional[str] = None, margin: float = 0.0) ->
Optional[Dict[str, Any]]:
+        """Compute the job's cost from ``usage`` × the pricebook + persist every cost column.
+        Returns the stored cost dict, or ``None`` on write failure. The caller gates this on
+        ``billing.enabled`` (default-OFF) ? nothing records until billing is turned on."""
+        rates = self.get_pricebook_rates(pricebook_version, gpu_type)
+        cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
+        if margin:
+            cost_usd = billing.apply_margin(cost_usd, margin)
+        bytes_out = int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
+        ok = self._execute_write(
+            "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?, gpu_active_seconds=?, "
+            "wall_seconds=?, bytes_out=?, gemini_cost_usd=?, pricebook_version=?, cost_usd=?, "
+            "cost_breakdown_json=? WHERE id = ?",
+            (owner_id, compute_backend, gpu_type, usage.get(billing.GPU_SECONDS),
+             usage.get(billing.WALL_SECONDS), bytes_out, usage.get(billing.GEMINI_USD),
+             pricebook_version, cost_usd, json.dumps(breakdown), job_id),
+            f"Failed to record cost for job {job_id}")
+        if not ok:
+            return None
+        return {"cost_usd": cost_usd, "cost_breakdown": breakdown,
+                "pricebook_version": pricebook_version}
+
+    def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
+        """The job's cost row (``None`` if the job doesn't exist; ``cost_usd`` is ``None``
until
+        recorded). ``cost_breakdown`` is the deserialized JSON breakdown."""
+        with self._connect() as conn:
+            row = conn.cursor().execute(
+                "SELECT id, video_id, owner_id, compute_backend, gpu_type, gpu_active_seconds,
"
+                "wall_seconds, bytes_out, gemini_cost_usd, pricebook_version, cost_usd, "
+                "cost_breakdown_json FROM jobs WHERE id = ?", (job_id,)).fetchone()
+        if not row:
+            return None
+        d = dict(row)
+        raw = d.pop("cost_breakdown_json", None)
+        d["cost_breakdown"] = json.loads(raw) if raw else {}
+        return d
+
+    def get_video_cost(self, video_id: str) -> Dict[str, Any]:
+        """Aggregate cost across a video's jobs: total ``cost_usd`` + the per-job rows. A video
```

```
is
+        1:1 with an infer job, but a re-ingest can produce several ? sum them."""
+        with self._connect() as conn:
+            rows = conn.cursor().execute(
+                "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM jobs "
+                "WHERE video_id = ?", (video_id,)).fetchall()
+        jobs: List[Dict[str, Any]] = []
+        total = 0.0
+        for r in rows:
+            cost = r["cost_usd"]
+            if cost is not None:
+                total += float(cost)
+            jobs.append({
+                "job_id": r["id"], "cost_usd": cost, "pricebook_version":
r["pricebook_version"],
+                "cost_breakdown": json.loads(r["cost_breakdown_json"]) if
r["cost_breakdown_json"] else {},
+            })
+        return {"video_id": video_id, "total_cost_usd": round(total, 6), "jobs": jobs}
+
     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2) -------------------------
     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
         """Attach/replace a job's JSON ExecutionPolicy."""
```

8. assistant (2026-06-21T11:10:45.788Z)

Now let me read the remaining files: config, main.py, job_worker.py, and the tests.

9. user (2026-06-21T11:10:46.977Z)

```
diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
index 1385c7a..d3ad660 100644
--- a/backend/api/job_worker.py
+++ b/backend/api/job_worker.py
@@ -88,6 +88,7 @@ def _run_claimed_job(job: Dict[str, Any]) -> None:
         if result.get("video_id"):
             db_instance.set_job_video_id(job_id, result["video_id"])
         db_instance.mark_job_done(job_id, result)
+        _maybe_record_job_cost(job, result)  # M8 cost ledger (default-OFF)
     except _main.JobWaiting as w:
         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
@@ -97,6 +98,32 @@ def _run_claimed_job(job: Dict[str, Any]) -> None:
         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


+def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
+    """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
+    ``config.billing.enabled``. When on, compute the job's cost from the usage the pipeline
reported
+    (``result['usage']`` = {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the
+    versioned pricebook, and persist it. With the placeholder pricebook every rate is 0 ? cost
0,
+    so turning billing on changes no user-facing number until the owner inserts real rates.
+
+    Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job`` is the
+    claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
+    import backend.main as _main
+
+    billing_cfg = getattr(_main.global_config, "billing", None)
+    if not billing_cfg or not getattr(billing_cfg, "enabled", False):
+        return
+    try:
+        usage = (result or {}).get("usage") or {}
+        _main.db_instance.record_job_cost(
```

```
+            job["id"], usage=usage, pricebook_version=billing_cfg.pricebook_version,
+            gpu_type=usage.get("gpu_type"), compute_backend=(result or
{}).get("compute_backend"),
+            owner_id=job.get("owner_id"), margin=billing_cfg.margin,
+        )
+        logger.info("Recorded cost for job %s (pricebook=%s).", job["id"],
billing_cfg.pricebook_version)
+    except Exception as e:  # never wedge a completed job on bookkeeping
+        logger.warning("Cost recording for job %s failed (non-fatal): %s", job.get("id"), e)
+
+
 def _drain_due_jobs() -> int:
     """One worker tick: claim and run every currently-runnable job (queued, or waiting whose
     `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
diff --git a/backend/config/models.py b/backend/config/models.py
index 92ad880..960f6c7 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -351,6 +351,21 @@ class DeploymentConfig(BaseModel):
     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""


+class BillingConfig(BaseModel):
+    """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
+
+    **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay NULL =
+    today's behaviour). When enabled, a job's cost is computed + stored in **USD** via the
versioned
+    ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
+    version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the owner
+    inserts a real, calibrated version and points ``pricebook_version`` at it."""
+
+    enabled: bool = False
+    pricebook_version: str = "placeholder-v0"   # the seeded $0 version; owner flips to a real
one
+    fx_inr_per_usd: float = 83.0                  # USD?INR display rate (Bangalore market)
+    margin: float = 0.0                            # resale markup (?0); 0 = bill at cost
+
+
 class AppConfig(BaseModel):
     """Root configuration object.

@@ -372,6 +387,7 @@ class AppConfig(BaseModel):
     logging: LoggingConfig = Field(default_factory=LoggingConfig)
     storage: StorageConfig = Field(default_factory=StorageConfig)
     deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
+    billing: BillingConfig = Field(default_factory=BillingConfig)

     # Allow extra keys during transition so existing config.json files do not break.
     model_config = {
diff --git a/backend/main.py b/backend/main.py
index 6b89f6d..bc2ac10 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -39,6 +39,7 @@ from backend.config import (  # new typed config
 from backend.results import Failure  # standardized error/result contract
 from backend.reporting import emit_indexer_report
 from backend import storage  # M2: URI-aware input acquisition (local = identity passthrough)
+from backend import billing  # M8: per-job cost ledger (USD internal / INR display)

 app = FastAPI(title="Sports Highlight Indexer API")

@@ -83,6 +84,7 @@ from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay
addr
```

```
     _get_executor,
     _job_to_request,
     _MAX_DRAIN_WAKE_SEC,
+    _maybe_record_job_cost,
     _run_claimed_job,
     _schedule_next_drain_if_waiting,
 )
@@ -306,6 +308,32 @@ def process_video(req: ProcessRequest):
     )


+@app.get("/api/jobs/{job_id}/cost")
+def get_job_cost(job_id: str):
+    """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of truth
(matches
+    GCP billing); ``cost_inr`` is a display conversion at the configured FX rate. ``cost_usd``
is
+    null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
+    cost = db_instance.get_job_cost(job_id)
+    if cost is None:
+        raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
+    fx = global_config.billing.fx_inr_per_usd
+    usd = cost.get("cost_usd")
+    cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
+    cost["fx_inr_per_usd"] = fx
+    return cost
+
+
+@app.get("/api/videos/{video_id}/cost")
+def get_video_cost(video_id: str):
+    """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` + ``total_cost_inr`` +
the
+    per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce several."""
+    agg = db_instance.get_video_cost(video_id)
+    fx = global_config.billing.fx_inr_per_usd
+    agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
+    agg["fx_inr_per_usd"] = fx
+    return agg
+
+
 @app.get("/api/jobs/{job_id}")
 def get_job(job_id: str):
     """Job state: {status, progress, result, reports}. `status` = execution state
```

10. user (2026-06-21T11:10:47.829Z)

```
diff --git a/tests/test_billing.py b/tests/test_billing.py
new file mode 100644
index 0000000..dee2101
--- /dev/null
+++ b/tests/test_billing.py
@@ -0,0 +1,157 @@
+"""Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
+
+Pure cost math + the v6 migration (cost columns + versioned pricebook) + record/get/aggregate +
+the default-OFF wiring. Cardinal invariant: with the seeded ``placeholder-v0`` pricebook (every
+rate 0.0) every cost is 0, and with ``billing.enabled=False`` NOTHING records (cost columns
stay
+NULL = today).
+"""
+from backend import billing
+from backend.config import AppConfig
+from backend.infrastructure.database import Database
+
+
```

```
+# ------------------------------------------------------------- pure cost module
+
+def test_compute_cost_zero_without_rates():
+    cost, bd = billing.compute_cost_usd({"gpu_seconds": 180, "egress_gb": 0.04}, {})
+    assert cost == 0.0 and bd == {"gpu_seconds": 0.0, "egress_gb": 0.0}
+
+
+def test_compute_cost_with_rates_and_gemini_passthrough():
+    usage = {"gpu_seconds": 180, "egress_gb": 0.04, "gemini_usd": 0.05}
+    rates = {"gpu_seconds": 0.0002, "egress_gb": 0.10}
+    cost, bd = billing.compute_cost_usd(usage, rates)
+    assert cost == round(180 * 0.0002 + 0.04 * 0.10 + 0.05, 6)  # 0.036 + 0.004 + 0.05
+    assert bd["gpu_seconds"] == 0.036 and bd["egress_gb"] == 0.004 and bd["gemini_usd"] == 0.05
+
+
+def test_apply_margin_clamps_and_to_inr():
+    assert billing.apply_margin(0.10, 0.20) == 0.12
+    assert billing.apply_margin(0.10, -5) == 0.10        # margin clamped to >= 0 (never below
cost)
+    assert billing.to_inr(1.0, 83.0) == 83.0
+
+
+# ------------------------------------------------------------- v6 migration + pricebook
seed
+
+def test_v6_migration_adds_cost_columns_and_seeds_placeholder(tmp_path):
+    db = Database(str(tmp_path / "m.db"))
+    with db._connect() as conn:
+        ver = conn.execute("PRAGMA user_version").fetchone()[0]
+        cols = {r[1] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
+        seeded = conn.execute(
+            "SELECT unit_price_usd FROM pricebook WHERE version='placeholder-v0'").fetchall()
+    assert ver >= 6
+    assert {"cost_usd", "gpu_active_seconds", "wall_seconds", "bytes_out", "owner_id",
+            "pricebook_version", "cost_breakdown_json", "compute_backend", "gpu_type"} <= cols
+    assert len(seeded) >= 4 and all(r[0] == 0.0 for r in seeded)  # placeholder = every rate $0
+
+
+def test_v6_migration_is_idempotent(tmp_path):
+    """Re-opening the DB re-runs the ladder guard (version already 6 ? no-op, no duplicate
seed)."""
+    path = str(tmp_path / "m.db")
+    Database(path)
+    db2 = Database(path)  # second open must not crash or duplicate the seed
+    with db2._connect() as conn:
+        n = conn.execute("SELECT COUNT(*) FROM pricebook WHERE
version='placeholder-v0'").fetchone()[0]
+    assert n == 4  # exactly the 4 seeded rows (INSERT OR IGNORE on the unique key)
+
+
+# ------------------------------------------------------------- pricebook rate resolution
+
+def _seed_real_pricebook(db, version="v1"):
+    with db._connect() as conn:
+        conn.executemany(
+            "INSERT INTO pricebook (version, resource, gpu_type, unit_price_usd, currency) "
+            "VALUES (?, ?, ?, ?, 'USD')",
+            [(version, "gpu_seconds", "L4", 0.0002, ), (version, "gpu_seconds", "T4", 0.0001,
),
+             (version, "egress_gb", "", 0.10, ), (version, "wall_seconds", "", 0.0, )])
+        conn.commit()
+
+
+def test_get_pricebook_rates_selects_by_gpu_type(tmp_path):
+    db = Database(str(tmp_path / "m.db"))
```

```
+    _seed_real_pricebook(db)
+    l4 = db.get_pricebook_rates("v1", "L4")
+    assert l4["gpu_seconds"] == 0.0002 and l4["egress_gb"] == 0.10
+    assert db.get_pricebook_rates("v1", "T4")["gpu_seconds"] == 0.0001
+    assert db.get_pricebook_rates("nonexistent", "L4") == {}  # unknown version ? no rates ?
cost 0
+
+
+# ------------------------------------------------------------------ record / get / aggregate
+
+def test_record_and_get_job_cost(tmp_path):
+    db = Database(str(tmp_path / "m.db"))
+    db.create_job("j1", "v.mp4")
+    db.set_job_video_id("j1", "vidA")
+    _seed_real_pricebook(db)
+    out = db.record_job_cost("j1", usage={"gpu_seconds": 180, "egress_gb": 0.04,
"wall_seconds": 200},
+                             pricebook_version="v1", gpu_type="L4",
compute_backend="cloud_run",
+                             owner_id="u1")
+    assert out["cost_usd"] == round(180 * 0.0002 + 0.04 * 0.10, 6)
+    cost = db.get_job_cost("j1")
+    assert cost["cost_usd"] == out["cost_usd"]
+    assert cost["gpu_type"] == "L4" and cost["owner_id"] == "u1" and cost["compute_backend"] ==
"cloud_run"
+    assert cost["bytes_out"] == int(0.04 * 1e9)
+    assert cost["cost_breakdown"]["gpu_seconds"] == 0.036
+
+
+def test_record_with_placeholder_pricebook_is_zero(tmp_path):
+    db = Database(str(tmp_path / "m.db"))
+    db.create_job("j1", "v.mp4")
+    out = db.record_job_cost("j1", usage={"gpu_seconds": 180},
pricebook_version="placeholder-v0",
+                             gpu_type="L4")
+    assert out["cost_usd"] == 0.0  # default placeholder ? $0, nothing user-facing changes
+
+
+def test_record_applies_margin(tmp_path):
+    db = Database(str(tmp_path / "m.db"))
+    db.create_job("j1", "v.mp4")
+    _seed_real_pricebook(db)
+    out = db.record_job_cost("j1", usage={"gpu_seconds": 100}, pricebook_version="v1",
gpu_type="L4",
+                             margin=0.20)
+    assert out["cost_usd"] == round(100 * 0.0002 * 1.20, 6)
+
+
+def test_get_job_cost_unknown_returns_none(tmp_path):
+    assert Database(str(tmp_path / "m.db")).get_job_cost("nope") is None
+
+
+def test_get_video_cost_aggregates_across_jobs(tmp_path):
+    db = Database(str(tmp_path / "m.db"))
+    for jid in ("j1", "j2"):
+        db.create_job(jid, "v.mp4")
+        db.set_job_video_id(jid, "vidA")
+    _seed_real_pricebook(db)
+    db.record_job_cost("j1", usage={"gpu_seconds": 100}, pricebook_version="v1", gpu_type="L4")
+    db.record_job_cost("j2", usage={"gpu_seconds": 50}, pricebook_version="v1", gpu_type="L4")
+    agg = db.get_video_cost("vidA")
+    assert agg["total_cost_usd"] == round(100 * 0.0002 + 50 * 0.0002, 6)
+    assert len(agg["jobs"]) == 2
+
+
```

```
+# ------------------------------------------------------------- default-OFF wiring
+
+def test_maybe_record_job_cost_noop_when_disabled(tmp_path, monkeypatch):
+    """DEFAULT-OFF: billing.enabled=False ? the hook records NOTHING (cost stays NULL)."""
+    import backend.main as bmain
+    from backend.api import job_worker
+
+    db = Database(str(tmp_path / "m.db"))
+    db.create_job("j1", "v.mp4")
+    monkeypatch.setattr(bmain, "db_instance", db, raising=False)
+    monkeypatch.setattr(bmain, "global_config", AppConfig(), raising=False)  #
billing.enabled=False
+    job_worker._maybe_record_job_cost({"id": "j1"}, {"usage": {"gpu_seconds": 180}})
+    assert db.get_job_cost("j1")["cost_usd"] is None  # nothing recorded
+
+
+def test_maybe_record_job_cost_records_when_enabled(tmp_path, monkeypatch):
+    import backend.main as bmain
+    from backend.api import job_worker
+
+    db = Database(str(tmp_path / "m.db"))
+    db.create_job("j1", "v.mp4")
+    monkeypatch.setattr(bmain, "db_instance", db, raising=False)
+    cfg = AppConfig.model_validate({"billing": {"enabled": True, "pricebook_version":
"placeholder-v0"}})
+    monkeypatch.setattr(bmain, "global_config", cfg, raising=False)
+    job_worker._maybe_record_job_cost({"id": "j1"}, {"usage": {"gpu_seconds": 180, "gpu_type":
"L4"}})
+    assert db.get_job_cost("j1")["cost_usd"] == 0.0  # recorded (0 with placeholder pricebook)
```

11. assistant (2026-06-21T11:10:51.426Z)

Now let me look closely at the cost-correctness lens. Let me examine the `get_pricebook_rates` behavior more carefully for the silent under-bill scenario, and the `record_job_cost` bytes_out round-trip. Let me also check the cost tests in test_api_endpoints.py.

12. user (2026-06-21T11:10:52.504Z)

```
diff --git a/tests/test_api_endpoints.py b/tests/test_api_endpoints.py
index 47d02e1..470c7b4 100644
--- a/tests/test_api_endpoints.py
+++ b/tests/test_api_endpoints.py
@@ -546,3 +546,49 @@ def test_cli_missing_local_returns_without_segmenting(tmp_path,
monkeypatch):
                                        str(tmp_path / "nope.mp4"), "--output-dir",
str(tmp_path)])
     m.main()
     assert called["seg"] is False
+
+
+# --- M8 cost-ledger endpoints (COMPUTE_DECOUPLED_SERVING M8, D8) ----------------------------
+
+def test_job_cost_endpoint_null_before_recording(env):
+    client, db, _tmp, _mp = env
+    db.create_job("j1", "v.mp4")
+    r = client.get("/api/jobs/j1/cost")
+    assert r.status_code == 200
+    body = r.json()
+    assert body["cost_usd"] is None and body["cost_inr"] is None   # nothing recorded yet
+    assert body["fx_inr_per_usd"] == 83.0                           # the default FX surfaced
+
+
+def test_job_cost_endpoint_unknown_404(env):
+    client, _db, _tmp, _mp = env
+    assert client.get("/api/jobs/nope/cost").status_code == 404
```

```
+
+
+def test_job_cost_endpoint_after_recording_shows_inr(env):
+    client, db, _tmp, _mp = env
+    db.create_job("j1", "v.mp4")
+    with db._connect() as conn:
+        conn.execute("INSERT INTO pricebook (version, resource, gpu_type, unit_price_usd,
currency) "
+                     "VALUES ('v1','gpu_seconds','L4',0.001,'USD')")
+        conn.commit()
+    db.record_job_cost("j1", usage={"gpu_seconds": 100}, pricebook_version="v1", gpu_type="L4")
+    body = client.get("/api/jobs/j1/cost").json()
+    assert body["cost_usd"] == 0.1
+    assert body["cost_inr"] == round(0.1 * 83.0, 2)   # USD?INR at the configured FX
+
+
+def test_video_cost_endpoint_aggregates(env):
+    client, db, _tmp, _mp = env
+    for jid in ("j1", "j2"):
+        db.create_job(jid, "v.mp4")
+        db.set_job_video_id(jid, "vidA")
+    with db._connect() as conn:
+        conn.execute("INSERT INTO pricebook (version, resource, gpu_type, unit_price_usd,
currency) "
+                     "VALUES ('v1','gpu_seconds','L4',0.001,'USD')")
+        conn.commit()
+    db.record_job_cost("j1", usage={"gpu_seconds": 100}, pricebook_version="v1", gpu_type="L4")
+    db.record_job_cost("j2", usage={"gpu_seconds": 50}, pricebook_version="v1", gpu_type="L4")
+    body = client.get("/api/videos/vidA/cost").json()
+    assert body["total_cost_usd"] == 0.15 and body["total_cost_inr"] == round(0.15 * 83.0, 2)
+    assert len(body["jobs"]) == 2
```

13. user (2026-06-21T11:10:53.303Z)

```
1       """Per-job cost ledger ? pure cost math (COMPUTE_DECOUPLED_SERVING M8, decision D8).
2
3       ``cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)``
4       (the formula from the archived 05-COST-ATTRIBUTION design). Cost is computed + stored in
**USD**
5       (matches GCP billing = one source of truth) and **displayed in INR** at a configurable
FX rate
6       (the Bangalore market). The pricebook is a versioned DB table
(``backend/infrastructure/database.py``);
7       this module is the pure arithmetic over a resolved ``{resource: unit_price_usd}`` rates
dict, so it
8       is unit-tested with zero IO and the DB layer owns the version/gpu_type lookup.
9
10      **Default-OFF:** with the seeded ``placeholder-v0`` pricebook (every rate ``0.0``) every
cost is
11      ``0.0`` ? nothing user-facing changes until the owner inserts a real, calibrated
pricebook version.
12      """
13
14      from __future__ import annotations
15
16      from typing import Dict, Mapping, Tuple
17
18      # Metered resource keys ? shared by the usage dict, the pricebook rows, and the
breakdown.
19      GPU_SECONDS = "gpu_seconds"     # GPU-active seconds (the big one) ? rate is per-second,
per gpu_type
20      WALL_SECONDS = "wall_seconds"   # total wall time (separate from GPU so idle/sync isn't
billed as GPU)
21      EGRESS_GB = "egress_gb"         # reel/byte egress in GB
22      GEMINI_USD = "gemini_usd"       # provider-reported cost, ALREADY in USD (a passthrough,
```

```
not rate-×)
23
24      # Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a
USD amount).
25      _RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
26
27
28      def compute_cost_usd(usage: Mapping[str, float],
29                           rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
30          """Return ``(cost_usd, breakdown)`` for one job.
31
32          ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps
a resource
33          key to its USD unit price (already resolved for this pricebook version + gpu_type by
the DB
34          layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
multiplied).
35          Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total
is ``0.0``.
36          """
37          breakdown: Dict[str, float] = {}
38          total = 0.0
39          for res in _RATE_MULTIPLIED:
40              amount = float(usage.get(res, 0.0) or 0.0)
41              rate = float(rates.get(res, 0.0) or 0.0)
42              cost = amount * rate
43              if amount or rate:                # record the line even at $0 so the breakdown
is transparent
44                  breakdown[res] = round(cost, 6)
45              total += cost
46          gemini = float(usage.get(GEMINI_USD, 0.0) or 0.0)
47          if gemini:
48              breakdown[GEMINI_USD] = round(gemini, 6)
49          total += gemini
50          return round(total, 6), breakdown
51
52
53      def apply_margin(cost_usd: float, margin: float) -> float:
54          """Resale price = ``cost_usd × (1 + margin)``. ``margin`` is clamped ? 0 (never
below cost)."""
55          return round(cost_usd * (1.0 + max(0.0, float(margin))), 6)
56
57
58      def to_inr(cost_usd: float, fx_inr_per_usd: float) -> float:
59          """Display conversion: USD ? INR at the configured FX rate (2 dp, the display
currency)."""
60          return round(float(cost_usd) * float(fx_inr_per_usd), 2)
61
```

14. assistant (2026-06-21T11:10:59.212Z)

Now let me focus on the cost-correctness lens deeply. Let me examine the key concern from sub-question (2): the silent under-bill when a pricebook version has `gpu_seconds` rows but not for the requested `gpu_type`. Let me trace the exact behavior in `get_pricebook_rates`.

Looking at lines in database.py:
```python
if res == billing.GPU_SECONDS:
    if row_gt == gt:        # the GPU rate for THIS gpu_type
        rates[res] = price
```

If the requested `gpu_type` (e.g. "A100") has no matching row, `rates["gpu_seconds"]` is never set, so `compute_cost_usd` defaults it to 0.0 ? GPU cost is silently $0. Let me verify the surrounding write path and whether usage carries `gpu_active_seconds` properly. Let me also

check the `bytes_out` round-trip precision and the `gpu_type=None` default-empty handling.

15. user (2026-06-21T11:11:00.381Z)

```
602         # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8)
----------------------------
603         def get_pricebook_rates(self, version: str, gpu_type: Optional[str] = None) ->
Dict[str, float]:
604             """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``.
The per-second
605             GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
``gpu_type=''``.
606             An unknown version ? ``{}`` (so every cost computes to 0.0)."""
607             gt = gpu_type or ""
608             rates: Dict[str, float] = {}
609             with self._connect() as conn:
610                 rows = conn.cursor().execute(
611                     "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE version
= ?",
612                     (version,)).fetchall()
613             for r in rows:
614                 res, row_gt, price = r["resource"], (r["gpu_type"] or ""),
float(r["unit_price_usd"])
615                 if res == billing.GPU_SECONDS:
616                     if row_gt == gt:        # the GPU rate for THIS gpu_type
617                         rates[res] = price
618                 else:
619                     rates[res] = price      # wall/egress: gpu_type-independent
620             return rates
621
622         def record_job_cost(self, job_id: str, *, usage: Dict[str, float],
pricebook_version: str,
623                             gpu_type: Optional[str] = None, compute_backend: Optional[str] =
None,
624                             owner_id: Optional[str] = None, margin: float = 0.0) ->
Optional[Dict[str, Any]]:
625             """Compute the job's cost from ``usage`` × the pricebook + persist every cost
column.
626             Returns the stored cost dict, or ``None`` on write failure. The caller gates
this on
627             ``billing.enabled`` (default-OFF) ? nothing records until billing is turned
on."""
628             rates = self.get_pricebook_rates(pricebook_version, gpu_type)
629             cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
630             if margin:
631                 cost_usd = billing.apply_margin(cost_usd, margin)
632             bytes_out = int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
633             ok = self._execute_write(
634                 "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?,
gpu_active_seconds=?, "
635                 "wall_seconds=?, bytes_out=?, gemini_cost_usd=?, pricebook_version=?,
cost_usd=?, "
636                 "cost_breakdown_json=? WHERE id = ?",
637                 (owner_id, compute_backend, gpu_type, usage.get(billing.GPU_SECONDS),
638                  usage.get(billing.WALL_SECONDS), bytes_out, usage.get(billing.GEMINI_USD),
639                  pricebook_version, cost_usd, json.dumps(breakdown), job_id),
640                 f"Failed to record cost for job {job_id}")
641             if not ok:
642                 return None
643             return {"cost_usd": cost_usd, "cost_breakdown": breakdown,
644                     "pricebook_version": pricebook_version}
645
646         def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
647             """The job's cost row (``None`` if the job doesn't exist; ``cost_usd`` is
``None`` until
```

```
648             recorded). ``cost_breakdown`` is the deserialized JSON breakdown."""
649             with self._connect() as conn:
650                 row = conn.cursor().execute(
651                     "SELECT id, video_id, owner_id, compute_backend, gpu_type,
gpu_active_seconds, "
652                     "wall_seconds, bytes_out, gemini_cost_usd, pricebook_version, cost_usd,
"
653                     "cost_breakdown_json FROM jobs WHERE id = ?", (job_id,)).fetchone()
654             if not row:
655                 return None
656             d = dict(row)
657             raw = d.pop("cost_breakdown_json", None)
658             d["cost_breakdown"] = json.loads(raw) if raw else {}
659             return d
660
661         def get_video_cost(self, video_id: str) -> Dict[str, Any]:
662             """Aggregate cost across a video's jobs: total ``cost_usd`` + the per-job rows.
A video is
663             1:1 with an infer job, but a re-ingest can produce several ? sum them."""
664             with self._connect() as conn:
665                 rows = conn.cursor().execute(
666                     "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM jobs "
667                     "WHERE video_id = ?", (video_id,)).fetchall()
668             jobs: List[Dict[str, Any]] = []
669             total = 0.0
670             for r in rows:
671                 cost = r["cost_usd"]
672                 if cost is not None:
673                     total += float(cost)
674                 jobs.append({
675                     "job_id": r["id"], "cost_usd": cost, "pricebook_version":
r["pricebook_version"],
676                     "cost_breakdown": json.loads(r["cost_breakdown_json"]) if
r["cost_breakdown_json"] else {},
677                 })
678             return {"video_id": video_id, "total_cost_usd": round(total, 6), "jobs": jobs}
679
680         # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
-------------------------
681         def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
```

16. assistant (2026-06-21T11:11:03.267Z)

Now let me verify a key financial-math concern with a quick numerical check: the `bytes_out` round-trip and the `int()` truncation behavior, plus whether `int(x * 1e9)` introduces float drift.

17. user (2026-06-21T11:11:06.323Z)

```
gb=0.04 -> bytes_out=40000000 -> back_to_gb=0.04
gb=0.1 -> bytes_out=100000000 -> back_to_gb=0.1
gb=1.5 -> bytes_out=1500000000 -> back_to_gb=1.5
gb=2.999999999 -> bytes_out=2999999999 -> back_to_gb=2.999999999
gb=0.123456789 -> bytes_out=123456789 -> back_to_gb=0.123456789
---
```

18. user (2026-06-21T11:11:09.558Z)

```
summed 0.171364
```

19. assistant (2026-06-21T11:11:14.123Z)

The `bytes_out` is a derived display field; cost uses `egress_gb` directly, so the `int()` truncation does not affect billing. USD source of truth is consistent.

Now let me examine the most important financial concern: the **silent under-bill** when GPU rate is missing for a `gpu_type`. Let me confirm by simulating the exact scenario, and also check whether `gpu_type=None` falls through to wall/egress-only (which is correct for CPU jobs but a silent $0 for a real GPU job with an unlisted type).

20. user (2026-06-21T11:11:21.694Z)

```
A100 job cost_usd = 0.004 breakdown= {'gpu_seconds': 0.0, 'egress_gb': 0.004}
   -> GPU 600s silently billed at $0; only egress 0.04*0.10=0.004 charged

gpu_type=None cost_usd = 0.004 breakdown= {'gpu_seconds': 0.0, 'egress_gb': 0.004}
```

21. assistant (2026-06-21T11:11:28.401Z)

Confirmed the silent under-bill. A real GPU job whose `gpu_type` isn't in the pricebook (typo, new GPU SKU like A100/H100, or `gpu_type=None` because the pipeline didn't report it) gets its GPU seconds billed at $0 ? and the breakdown shows `gpu_seconds: 0.0`, indistinguishable from a legitimately-free run. This is the most material money-math issue. The prompt explicitly asks whether this is "right behaviour or a silent under-bill" ? it is a silent under-bill.

Let me now verify a couple more lens items: the `margin` gating (`if margin:` ? what about negative margins passed through), and whether `apply_margin` is applied BEFORE breakdown so breakdown no longer sums to cost_usd (a reconciliation concern). Let me also check the `to_inr` rounding consistency between job-level and video-level.

22. user (2026-06-21T11:11:33.353Z)

```
pre-margin cost 0.02 breakdown {'gpu_seconds': 0.02} sum(bd)= 0.02
post-margin cost_usd stored = 0.024 but breakdown still sums to 0.02
   -> breakdown no longer reconciles to stored cost_usd when margin>0
```

23. user (2026-06-21T11:11:36.411Z)

```
per-job INR (0.004 usd) = 0.33 -> two jobs sum displayed per-job = 0.66
video-level INR on summed USD 0.008 = 0.66
   (USD is source of truth; INR display-only, so this rounding skew is acceptable per contract)
```

24. assistant (2026-06-21T11:11:43.557Z)

Two real findings so far:
1. Silent GPU under-bill (the big one).
2. When `margin > 0`, the stored `cost_usd` includes margin but `cost_breakdown_json` does NOT ? the breakdown lines no longer sum to the stored total, so the audit trail (the whole point of a versioned, auditable ledger) is internally inconsistent.

INR rounding skew is within-contract (USD is source of truth, INR display-only), so not a finding.

Now let me verify the gemini passthrough is correct (sub-question 1) and check one subtle thing: whether `gemini_usd` margin should apply. The margin is applied to the entire `cost_usd` which includes the gemini passthrough. That means you're marking up Gemini's already-USD cost too ? which may be intended (resale markup on total) or a double-charge concern. Let me also confirm the `gemini` cost is NOT rate-multiplied (it's correct ? added verbatim). Let me check whether margin-on-gemini is a defensible resale model vs an error.

25. user (2026-06-21T11:11:46.693Z)

```
cost 0.07 bd {'gpu_seconds': 0.02, 'gemini_usd': 0.05}
gemini passthrough OK (added, not multiplied)
margin on total incl gemini: 0.084 (gemini 0.05 -> contributes 0.06 to resale)
```

26. assistant (2026-06-21T11:11:50.058Z)

Gemini passthrough is correct (added, not multiplied). Margin-on-gemini marks up the resale

price of a passthrough cost ? this is a defensible resale model (markup on total cost-of-goods), not a money-math error, so not a finding.

Now let me run the billing tests to confirm the suite is green and my understanding matches actual behavior.

27. user (2026-06-21T11:11:58.846Z)

```
.............                                                            [100%]
13 passed in 5.82s
```

28. assistant (2026-06-21T11:12:06.447Z)

All 13 billing tests pass. Now let me double check one nuance in `get_pricebook_rates`: the prompt notes "wall/egress rates carry gpu_type=''". But the loop sets `rates[res] = price` for any non-GPU resource regardless of its `row_gt`. If a pricebook erroneously had a wall_seconds row with a non-empty gpu_type, it'd still be picked up. Minor, not a real money error given the seed/contract. Let me verify the partial-index and migration idempotency don't double-seed, and confirm `gpu_active_seconds` column vs the `gpu_seconds` usage key naming (the column is `gpu_active_seconds`, usage key is `gpu_seconds` ? consistent because `record_job_cost` maps `usage.get(billing.GPU_SECONDS)` ? `gpu_active_seconds` column). That's fine.

Let me confirm there's no issue with the `compute_cost_usd` summing un-rounded `cost` into `total` while storing rounded breakdown lines ? a potential breakdown-vs-total reconciliation drift independent of margin.

29. user (2026-06-21T11:12:10.648Z)

```
cost 2e-06 bd {'gpu_seconds': 0.0, 'wall_seconds': 0.0, 'egress_gb': 0.0} sum(bd) 0.0
5e-7 round6 = 0.0
```

30. assistant (2026-06-21T11:12:15.869Z)

Confirmed a second reconciliation gap: the `total` sums **un-rounded** per-line costs and rounds once at the end, while each `breakdown` line is rounded **independently** at 6dp. At sub-micro-dollar rates these can disagree (stored `cost_usd = 0.000002` but the breakdown sums to `0.0`). This is the same class of issue as the margin one ? the breakdown doesn't reconcile to the stored total. In practice this is negligible at realistic rates (sub-$0.000001 line items), so it's a minor/nit, whereas the margin case produces a material discrepancy (20% off) and is more clearly a defect.

I have enough grounded findings. Let me compile the structured output.

31. user (2026-06-21T11:12:37.809Z)

Structured output provided successfully

32. assistant (2026-06-21T11:12:49.660Z)

Review complete. I inspected the M8 cost-ledger diff (`git diff origin/master...HEAD`), read `backend/billing.py`, the `_migrate_to_v6` + cost helpers in `backend/infrastructure/database.py`, `BillingConfig`, the `/cost` endpoints in `backend/main.py`, `_maybe_record_job_cost` in `backend/api/job_worker.py`, and both test files; the 13 billing tests pass locally.

Three grounded findings under the cost-correctness lens (full detail in the structured output):

1. **major ? silent GPU under-bill** (`database.py:613-620` + `billing.py:39-45`): a real GPU job whose `gpu_type` isn't a row in the active pricebook version (new SKU, typo, or `gpu_type=None`) gets its GPU seconds ? the dominant cost ? billed at $0, with `breakdown={'gpu_seconds':0.0}` indistinguishable from a legitimately-free run. Reproduced: A100/600s against an L4/T4-only pricebook ? only egress charged. Missing *amount*?0 is safe; missing *rate* for a present amount is a silent pricing gap that should be surfaced/logged, not zeroed.

2. **minor ? breakdown doesn't reconcile to total when margin > 0** (`database.py:629-639`): `cost_usd` is overwritten with the margined value but `cost_breakdown_json` keeps the pre-margin lines, so the auditable ledger's line items no longer sum to the headline number.

3. **nit ? sub-micro-dollar rounding drift** (`billing.py:39-50`): total rounds once over un-rounded lines while each breakdown line rounds independently; harmless at real GCP rates.

Verified as **correct / not findings**: gemini_usd is a true passthrough (added, not rate-multiplied); missing amounts/rates default safely to 0; `bytes_out = int(egress_gb * 1e9)` is a derived display field that does not feed cost (cost uses `egress_gb` directly), so its truncation can't drift a bill; `apply_margin` clamps margin ? 0; `to_inr` is display-only with USD as source of truth (job-vs-video INR rounding skew is within the contract); `get_video_cost` correctly skips NULL `cost_usd` and sums across re-ingest jobs.